INDEX

S.No.	Title for Practicals	Page No.
1	Write a program in Python to insert elements in an Array	2
2	Write a program in Python to delete Elements in an Array	4
3	Write a program in Python to find the product of two matrices	5
4	Sum of Diagonal and Non-Diagonal Elements in Matrix	7
5	Marks Calculation	9
6	Operations of stacks and its Applications	12
7	General and Circular Queue Operations	21
8	Infix to Postfix Expression	28
9	Quick Sort	31

Q.1. Write a program in Python to Insert an element in the Array.

→Arrays are the special Linear Data Structures that we can use to store any amount of homogenous (same set of elements) data elements in a contiguous and sequential order.

If you were to use any old programming language, the array length is usually fixed in size.

Here in Python, the array size can either be fixed or dynamic, as in, the array length is calculated while the user is putting the values when the program is executing.

This dynamic feature takes away that problem of fixed array length and not being able to insert more elements to the Data Structure.

Here is how the insertion in an array looks like:

main.py

```python
print("\n ----------------------------------------")
print("\n This program is made by AMRAPALY SARKAR")
print("\n ----------------------------------------")

# 1. Take array elements from the user
user_input = input("Enter array elements separated by spaces: ")
arr = user_input.split()  # Converts input string into a list

print("Original Array:", arr)

# 2. Take the element to insert and the position (index)
item_to_insert = input("Enter the element you want to insert: ")
index = int(input("Enter the index (position) where you want to insert it: "))

# 3. Insert the element using insert()
arr.insert(index, item_to_insert)

print("\n Updated Array after insertion:", arr)
```

Code for Insertion Array

1

```
 ------------------------------------

 This program is made by AMRAPALY SARKAR

 ------------------------------------
Enter array elements separated by spaces: 30 60 90 120 150 180 210 240 270 300
Original Array: ['30', '60', '90', '120', '150', '180', '210', '240', '270', '300']
Enter the element you want to insert: 220
Enter the index (position) where you want to insert it: 7

 Updated Array after insertion: ['30', '60', '90', '120', '150', '180', '210', '220', '240', '270', '300']
```

This is how insertion in array works in Python.

Q.2. Write a program in Python to delete an element from array.

→Arrays are a Linear Data Structure that stores the data elements in a sequential and contiguous manner.

Now, the program to delete an element from the array:

main.py

```
print("\n ----------------------------------------")
print("\n This program is made by AMRAPALY SARKAR")
print("\n ----------------------------------------")

# 1. Take array elements from the user
user_input = input("Enter array elements separated by spaces: ")
arr = user_input.split()  # Converts input string into a list

print("Original Array:", arr)

# 2. Take the element the user wants to delete
item_to_delete = input("Enter the element you want to delete: ")

# 3. Check and delete the element
if item_to_delete in arr:
    arr.remove(item_to_delete)
    print("\n Updated Array after deletion:", arr)
else:
    print("\n Item not found in the array!")
```

Let's see how it works:

```
 ----------------------------------------

 This program is made by AMRAPALY SARKAR

 ----------------------------------------
Enter array elements separated by spaces: 10 20 30 40 50 60
Original Array: ['10', '20', '30', '40', '50', '60']
Enter the element you want to delete: 20

 Updated Array after deletion: ['10', '30', '40', '50', '60']
```

Q.3. Write a program in Python to find the product of two matrices.

→This program shows how to get the product of two matrices:

main.py

```
print("\n ---------------------------------------")
print("\n This program is made by AMRAPALY SARKAR")
print("\n ---------------------------------------")

# Let the user choose the size (e.g., 2 for 2x2, 3 for 3x3, etc.)
n = int(input("Enter the size of the square matrix (n): "))

# Input Matrix A
print(f"\nEnter elements for Matrix A ({n}x{n}):")
A = []
for i in range(n):
    row = [int(x) for x in input(f"Row {i + 1}: ").split()]
    A.append(row)

# Input Matrix B
print(f"\nEnter elements for Matrix B ({n}x{n}):")
B = []
for i in range(n):
    row = [int(x) for x in input(f"Row {i + 1}: ").split()]
    B.append(row)

# Create a blank result matrix filled with zeros
result = [[0 for _ in range(n)] for _ in range(n)]

# Multiply Matrix A and Matrix B
for i in range(n):
    for j in range(n):
        for k in range(n):
            result[i][j] += A[i][k] * B[k][j]

# Display the final product
print("\n--- Product Matrix (A x B) ---")
for row in result:
    print(row)
```

Let us see how the program works.

```
 ---------------------------------------

 This program is made by AMRAPALY SARKAR

 ---------------------------------------
Enter the size of the square matrix (n): 3

Enter elements for Matrix A (3x3):
Row 1: 1 1 3
Row 2: 2 1 1
Row 3: 3 2 1

Enter elements for Matrix B (3x3):
Row 1: 1 3 3
Row 2: 1 4 2
Row 3: 1 1 1

--- Product Matrix (A × B) ---
[5, 10, 8]
[4, 11, 9]
[6, 18, 14]
```

Thus, this is the program to calculate the product of two matrices.

Q.4. Write a program in Python to find the sum of diagonal elements as well as the non-diagonal elements of a matrix.

→ Here is a program to perform the sum of diagonal elements in a matrix and also calculating the sum of the non-diagonal elements in a matrix.

main.py

```
print("\n ------------------------------------")
print("\n This program is made by AMRAPALY SARKAR")
print("\n ------------------------------------")

# First, I am getting the matrix size
n = int(input("Enter the size of the square matrix (n): "))
matrix = []

# Then, I am getting the inputs for matrix rows
print("Enter each row separated by spaces:")
for i in range(n):
    row = [int(x) for x in input(f"Row {i + 1}: ").split()]
    matrix.append(row)

diagonal_sum = 0 # Initialised the sum with 0 for Diagonal
non_diagonal_sum = 0 # Initialised the sum for Non - Diagonal elements as 0

# Here, we are calculating the sums
for i in range(n):
    for j in range(n):
        if i == j:
            diagonal_sum += matrix[i][j]      # Diagonal (e.g., [0][0], [1][1])
        else:
            non_diagonal_sum += matrix[i][j]  # Everything else

print("\n--- Results ---")
print("Sum of Diagonal Elements     :", diagonal_sum)
print("Sum of Non-Diagonal Elements :", non_diagonal_sum)
```

Let us see how this program works.

```
 ---------------------------------------

 This program is made by AMRAPALY SARKAR

 ---------------------------------------
Enter the size of the square matrix (n): 3
Enter each row separated by spaces:
Row 1: 1 2 3
Row 2: 3 4 5
Row 3: 5 6 7

--- Results ---
Sum of Diagonal Elements     : 12
Sum of Non-Diagonal Elements : 24
```

Thus, this is our program for finding the sum of:

- Diagonal
- Non-Diagonal

Q.5. Write a program in Python to enter marks of 5 students for 3 subjects each and find the percentages of students marks.

→This program shows the results of 5 students in 3 different subjects each:

main.py

```
print("\n ----------------------------------------")
print("\n This program is made by AMRAPALY SARKAR")
print("\n ----------------------------------------")

# Loop for 5 students
for i in range(1, 6):
    print(f"--- Student {i} ---")

    # Taking input for 3 subjects
    sub1 = float(input("Enter marks for Subject 1: "))
    sub2 = float(input("Enter marks for Subject 2: "))
    sub3 = float(input("Enter marks for Subject 3: "))

    # Calculating total and percentage (out of 300 total marks)
    total_marks = sub1 + sub2 + sub3
    percentage = (total_marks / 300) * 100

    # Displaying the result for the student
    print(f"Total Marks: {total_marks} / 300")
    print(f"Percentage: {percentage:.2f}%\n")
```

Let us see how this program works.

```
 ---------------------------------------

 This program is made by AMRAPALY SARKAR

 ---------------------------------------
--- Student 1 ---
Enter marks for Subject 1: 80
Enter marks for Subject 2: 86
Enter marks for Subject 3: 83
Total Marks: 249.0 / 300
Percentage: 83.00%

--- Student 2 ---
Enter marks for Subject 1: 78
Enter marks for Subject 2: 85
Enter marks for Subject 3: 76
Total Marks: 239.0 / 300
Percentage: 79.67%

--- Student 3 ---
Enter marks for Subject 1: 90
Enter marks for Subject 2: 98
Enter marks for Subject 3: 88
Total Marks: 276.0 / 300
Percentage: 92.00%
```

```
--- Student 4 ---
Enter marks for Subject 1: 74
Enter marks for Subject 2: 73
Enter marks for Subject 3: 77
Total Marks: 224.0 / 300
Percentage: 74.67%

--- Student 5 ---
Enter marks for Subject 1: 89
Enter marks for Subject 2: 95
Enter marks for Subject 3: 92
Total Marks: 276.0 / 300
Percentage: 92.00%
```

Thus, this is a program for calculating 5 students' marks in 3 different subjects, with the maximum marks being 100.

Q.6. Write a program in Python to perform the following Stack operations:

- Basic Operations:
  - ➔ Insertion
  - ➔ Deletion
  - ➔ Overflow Condition
  - ➔ Underflow Condition
- Reverse of a string.
- Parenthesis Matching.

→ A Stack is a Linear Data Structure, following the principle of LIFO (Last In, First Out), which means, that the element that is inserted into the stack at the very last, will be the first one to 'pop' or delete from the stack.

The main focus for the first part of our program is:

- Basic Operations on Stack:

  The program (as shown in the screenshots below) compiles all of the basic operations that you can perform in the Stack.

main.py

```
print("\n ---------------------------------------")
print("\n This program is made by AMRAPALY SARKAR.")
print("\n ---------------------------------------")

def stack_menu(): #I am defining the entire logic for this stack operation program in stack_menu
    #Here, the stack is initially empty
    stack = []
    capacity = 0

    while True: #I am going to give option for the user so that they can choose what operations they want to perform
        print("\n--- STACK OPERATIONS MENU ---")
        print("1. Create Stack") #Creation of the stack
        print("2. Push") #Inserting an element into the stack
        print("3. Pop") #Deleting an element from the stack
        print("4. Check Overflow") #This condition will be shown when there is no more space for insertion in the stack, causing and overflow
        print("5. Check Underflow") #This condition will be shown when the stack is already empty and deletion is performed
        print("6. Exit") #Terminating the program with the break operation

        choice = input("Select an option from (1-6): ").strip()

        if choice == "1": #If user chose 1, stack will be created with the following functions defined under the condition
            try:
                capacity = int(input("Enter max capacity for the stack: "))
                stack = []
                print(f"Stack created with maximum capacity of {capacity}.")
            except ValueError:
                print("Invalid capacity. Please enter an integer.")

        elif choice == "2": #If user chooses 2, they can perform insertion into the stack, one element at a time
            if capacity == 0 and not stack:
                print("Please create a stack first (Option 1).")
            elif len(stack) >= capacity:
                print("Stack Overflow! Cannot push onto a full stack.")
            else:
```

main.py

```
                val = input("Enter value to push: ")
                stack.append(val)
                print(f"Pushed '{val}'. Current stack: {stack}")

        elif choice == "3": #If user chooses 3, Deletion will be performed, one element at a time
            if len(stack) == 0:
                print("Stack Underflow! Cannot pop from an empty stack.")
            else:
                popped_item = stack.pop()
                print(f"Popped '{popped_item}'. Current stack: {stack}")

        elif choice == "4": #The user can check if the stack is completely full or not
            if capacity == 0:
                print("Stack has not been created yet.")
            elif len(stack) >= capacity:
                print(f"Overflow status: TRUE (Stack is full: {len(stack)}/{capacity})")
            else:
                print(f"Overflow status: FALSE (Available space: {capacity - len(stack)})")

        elif choice == "5": #The user can check if the stack completely empty or has elements in the data structure
            if len(stack) == 0:
                print("Underflow status: TRUE (Stack is completely empty)")
            else:
                print(f"Underflow status: FALSE (Stack contains {len(stack)} items)")

        elif choice == "6": #User can terminate or exit this program using the choice '6'
            print("Exiting program.")
            break

        else: #User chose something else instead of choosing from 1-6 options
            print("Invalid choice. Please pick between 1 and 6.")

```

```
        else: #User chose something else instead of choosing from 1-6 options
            print("Invalid choice. Please pick between 1 and 6.")


stack_menu() #This will basically show the menu for the operations you will perform in the stack. We already defined this in the above codes.
```

Now, let us create a stack:

```
 ----------------------------------------

 This program is made by AMRAPALY SARKAR.

 ----------------------------------------

--- STACK OPERATIONS MENU ---
1. Create Stack
2. Push
3. Pop
4. Check Overflow
5. Check Underflow
6. Exit
Select an option from (1-6): 1
Enter max capacity for the stack: 5
Stack created with maximum capacity of 5.
```

Our stack with the capacity to hold 5 data elements is created!

Let us move on to Insertion:

```
--- STACK OPERATIONS MENU ---
1. Create Stack
2. Push
3. Pop
4. Check Overflow
5. Check Underflow
6. Exit
Select an option from (1-6): 2
Enter value to push: 60
Pushed '60'. Current stack: ['20', '30', '40', '50', '60']
```

We have inserted 5 elements: [20, 30, 40, 50, 60], into the stack, using the push operation. The stack is full.

Let us check for overflow:

```
--- STACK OPERATIONS MENU ---
1. Create Stack
2. Push
3. Pop
4. Check Overflow
5. Check Underflow
6. Exit
Select an option from (1-6): 4
Overflow status: TRUE (Stack is full: 5/5)
```

The stack is overflowing, and will not store anymore elements as the maximum capacity for the stack was 5 and we inserted 5 elements in the stack.

```
--- STACK OPERATIONS MENU ---
1. Create Stack
2. Push
3. Pop
4. Check Overflow
5. Check Underflow
6. Exit
Select an option from (1-6): 2
Stack Overflow! Cannot push onto a full stack.
```

As we can see, when we tried to insert an element, it printed out 'Stack Overflow!' Thus, no more elements can be stored in the stack.

Let us delete some elements (Pop Operation):

```
--- STACK OPERATIONS MENU ---
1. Create Stack
2. Push
3. Pop
4. Check Overflow
5. Check Underflow
6. Exit
Select an option from (1-6): 3
Popped '60'. Current stack: ['20', '30', '40', '50']
```

When we performed deletion on the Stack, ‘60’ popped out, following the LIFO principle. ‘60’ was the last element that was pushed into the stack, so, according to LIFO, ‘60’ will also be the first one to be deleted or ‘popped’ out of the stack.

Checking Underflow:

```
Select an option from (1-6): 3
Popped '20'. Current stack: []
```

Just like how we popped ‘60’ out of the stack, we popped the other elements in the stack, following that same process. We can see that the stack is now empty. So, if you try to delete again, this message will pop up:

```
--- STACK OPERATIONS MENU ---
1. Create Stack
2. Push
3. Pop
4. Check Overflow
5. Check Underflow
6. Exit
Select an option from (1-6): 3
Stack Underflow! Cannot pop from an empty stack.
```

The message ‘Stack Underflow!’ will appear on the screen if you try to delete or perform the ‘pop’ operation.

Thus, these were some of the basic operations in a stack. Now, let us move on to the next ones.

- Reverse of a String

  Here, we are writing a program to reverse any string that is inputted by the user.

main.py

```python
print("\n ------------------------------------")
print("\n This program is made by AMRAPALY SARKAR")
print("\n ------------------------------------")

def reverse_string_with_stack(text): # This is the core logic for our string reversal program. It is the defined class for this program.
    #We are using a stack
    stack = []

    #Step 1: Push every character onto the stack
    for char in text: # I am using a loop for putting every character of the string into the stack
        stack.append(char)

    # Step 2: Pop characters off the stack to build the reversed string
    reversed_text = "" # Here, the stack will pop every character and store it into this variable
    while len(stack) > 0: # While the length of stack is greater than 0, it will keep popping the characters until false
        reversed_text += stack.pop()

    return reversed_text


def string_reversal_menu():
    # Menu interface to take user input and reverse strings
    while True:
        print("\n--- STRING REVERSAL USING STACK ---")
        print("1. Reverse a custom string")
```

```python
def string_reversal_menu():
    # Menu interface to take user input and reverse strings
    while True:
        print("\n--- STRING REVERSAL USING STACK ---")
        print("1. Reverse a custom string")
        print("2. Exit")

        choice = input("Select an option (1-2): ").strip()

        if choice == "1":
            user_input = input("\nEnter the string you want to reverse: ")

            if not user_input: #if user didn't put any string value, then we print this message
                print("String cannot be empty! Please try again.")
                continue

            reversed_result = reverse_string_with_stack(user_input)

            print("\n" + "=" * 40)
            print(f"Original Input : {user_input}")
            print(f"Reversed Output: {reversed_result}")
            print("=" * 40)

        elif choice == "2": # If user chose option '2', the program terminates or ends, thus, exiting the program
            print("Exiting String Reversal program.")
            break
        else:
            print("Invalid choice! Please select 1 or 2.")


if __name__ == "__main__": # The program entry point starts here
    string_reversal_menu()
```

Now, let us see what happens when the user inputs a string into this program:

```
 ----------------------------------------

 This program is made by AMRAPALY SARKAR

 ----------------------------------------

--- STRING REVERSAL USING STACK ---
1. Reverse a custom string
2. Exit
Select an option (1-2): 1

Enter the string you want to reverse: Data Structures

========================================
Original Input : Data Structures
Reversed Output: serutcurtS ataD
========================================
```

We inputted the string “Data Structures”, which got reversed into the string that is shown in the above picture.

Thus, this is how a stack is used to reverse any string.

Let us now move on to the next one:

- Parenthesis Matching

  This program shows how you can ensure that any equation with multiple parenthesis in it has both the opening and closing parenthesis (or brackets).

main.py

```python
print("\n ---------------------------------------")
print("\n This program is made by AMRAPALY SARKAR")
print("\n ---------------------------------------")

def is_balanced(s: str) -> bool: # This is the core function of the program that evaluates every string value, character by character
    stack = []
    bracket_map = {")": "(", "}": "{", "]": "["} # I mapped out the parenthesis pairings using Dictionary.
    opening_brackets = set(bracket_map.values()) #This is where the bracket's or parenthesis openings are easily looked up, when the program is executing.

    for char in s: # I am using a loop to evaluate the whole expression, which will be in string, to start with parenthesis matching
        if char in opening_brackets: # When bracket is encountered, it will be oushed into stack
            stack.append(char)
        elif char in bracket_map: # Checking whether the bracket is a closing bracket
            if not stack or stack.pop() != bracket_map[char]:
                return False

    return len(stack) == 0

user_input = input("Enter an expression: ") #Taking the input of the user here

if is_balanced(user_input): #If the expression has balanced pairings of parenthesis, then prints output 'Balanced'
    print("Balanced brackets")
else: # or just prints unbalanced
    print("Unbalanced brackets")
```

Now, let's see how this program works.

```
 ---------------------------------------

 This program is made by AMRAPALY SARKAR

 ---------------------------------------
Enter an expression: {[a*(b+c)]*d}
Balanced brackets
```

We inputted the expression, '{[a*(b+c)]*d}', then checked if it has matching parenthesis, as in, has both the closing and opening brackets. It has all the matching brackets, thus, it is Balanced.

Now, let us see when it is the other way around.

```
 ---------------------------------------

 This program is made by AMRAPALY SARKAR

 ---------------------------------------
Enter an expression: [(a+b)*c
Unbalanced brackets
```

We took '[(a+b)*c', as the expression. We checked and got unbalanced, which is correct, as it does not contain the closing square bracket.

Thus, these were some applications and operations performed using a stack data structure.

Q.7. Write a program in Python to perform the following:

- General Queue Operations
    a. Insertion
    b. Deletion
- Circular Queue Operations
    a. Insertion
    b. Deletion

→ Queues is a Linear Data Structure that uses the FIFO(First In, First Out) principle.

There two main types of queues that we are focusing on here:

I. Linear Queue
II. Circular Queue

Here is the code for the program: (Next Page)

main.py

```python
print("\n ------------------------------")
print("\n Program made by AMRAPALY SARKAR")
print("\n ------------------------------")

MAX = 20 #Defines the maximum capacity/size allowed for both queues

linear_queue = [None] * MAX #This code block contains a list of 20 with no element currently held by the program is defined here
#We are tracking the front and the rear of the queues
l_front = -1
l_rear = -1


def linear_enqueue(): #This is the definition class for the insertion of the elements inputted by the user into the queue
    global l_front, l_rear

    if l_rear == MAX - 1:
        print("\nYour Queue is overflowing!")
        return

    value = int(input("Enter the values for your queue: "))

    if l_front == -1:
        l_front = 0

    l_rear += 1
    linear_queue[l_rear] = value

    print(value, "is now in the Queue.")


def linear_dequeue(): #This class definition is for the deletion of elements from the queue
    global l_front, l_rear
```

main.py

```python
    if l_front == -1:
        print("\nYour queue is full! Queue Underflow!")
        return

    value = linear_queue[l_front]
    print(value, "is deleted from Queue.")

    l_front += 1

    if l_front > l_rear:
        l_front = -1
        l_rear = -1


def linear_display(): #This class definition is focusing on the queue's status, whether it is full or empty
    if l_front == -1:
        print("\nYour queue is empty.")
        return

    print("\nYour queue:", end=" ")

    for i in range(l_front, l_rear + 1):
        print(linear_queue[i], end=" ")

    print()


def linear_peek(): #This class definition displays the elements currently sitting at the front part of the queue
    if l_front == -1:
        print("\nYour queue is empty.")
    else:
        print("\nFront element:", linear_queue[l_front])
```

main.py

```
67  circular_queue = [None] * MAX #I am initialing my Circular Queue in this step
68  #The front and rear of the circular queue is initialised here
69  c_front = -1
70  c_rear = -1
71
72
73  def circular_enqueue(): #I am defining the insertion process in our queue structure
74      global c_front, c_rear
75
76      if (c_rear + 1) % MAX == c_front:
77          print("\nCircular Queue Overflow!")
78          return
79
80      value = int(input("Enter values for your queue: "))
81
82      if c_front == -1:
83          c_front = 0
84          c_rear = 0
85      else:
86          c_rear = (c_rear + 1) % MAX
87
88      circular_queue[c_rear] = value
89
90      print(value, "inserted into Circular Queue.")
91
92
93  def circular_dequeue(): #I am defining the deletion of elements from the queue using the dequeue class
94      global c_front, c_rear
95
96      if c_front == -1:
97          print("\nCircular Queue Underflow!")
98          return
99
```

main.py

```
100     value = circular_queue[c_front]
101
102     print(value, "deleted from Circular Queue.")
103
104     if c_front == c_rear:
105         c_front = -1
106         c_rear = -1
107     else:
108         c_front = (c_front + 1) % MAX
109
110
111 def circular_display(): #Here, I am checking if the circular queue is empty or full
112     if c_front == -1:
113         print("\nCircular Queue is empty.")
114         return
115
116     print("\nYour Circular Queue is:", end=" ")
117
118     i = c_front
119
120     while True:
121         print(circular_queue[i], end=" ")
122
123         if i == c_rear:
124             break
125
126         i = (i + 1) % MAX
127
128     print()
129
130
131 def circular_peek(): #The elements of the circular queue's front is getting displyed without any deletion
132     if c_front == -1:
```

main.py

```
110
111 def circular_display(): #Here, I am checking if the circular queue is empty or full
112     if c_front == -1:
113         print("\nCircular Queue is empty.")
114         return
115
116     print("\nYour Circular Queue is:", end=" ")
117
118     i = c_front
119
120     while True:
121         print(circular_queue[i], end=" ")
122
123         if i == c_rear:
124             break
125
126         i = (i + 1) % MAX
127
128     print()
129
130
131 def circular_peek(): #The elements of the circular queue's front is getting displyed without any deletion
132     if c_front == -1:
133         print("\nCircular Queue is empty.")
134     else:
135         print("\nFront element:", circular_queue[c_front])
136
137 while True: #This is where I provide the user with different options for Linear and Circular Queues
138
139     print("1. Linear Queue")
140     print("2. Circular Queue")
141     print("3. Exit")
```

main.py

```
140     print("2. Circular Queue")
141     print("3. Exit")
142
143     choice = int(input("Enter your choice: "))
144
145     if choice == 1:
146
147         while True: #If options are chosen from the following list, that particular option will be performed on the linear queue
148
149             print("\n---------- LINEAR QUEUE ----------")
150             print("1. Insert element in the queue.")
151             print("2. Delete element from the queue.")
152             print("3. Display my queue")
153             print("4. Show element at the front.")
154             print("5. Back to Main Menu")
155
156             operation = int(input("What do you want to do in your queue? Choose from the options (1-4): "))
157
158             if operation == 1:
159                 linear_enqueue()
160
161             elif operation == 2:
162                 linear_dequeue()
163
164             elif operation == 3:
165                 linear_display()
166
167             elif operation == 4:
168                 linear_peek()
169
170             elif operation == 5:
171                 break
172
```

main.py

```
            elif operation == 5:
                break

            else:
                print("Invalid choice!")

    elif choice == 2:

        while True: #Again, the user can choose from 1-5 for the opations they want to perform on the Circular Queue this time around

            print("\n--------- CIRCULAR QUEUE ---------")
            print("1. Insert an element in your circular queue.")
            print("2. Delete an element from your circular queue.")
            print("3. Display")
            print("4. Show front element in the circular queue.")
            print("5. Back to Main Menu")

            operation = int(input("What do you want to do in your queue? Choose from the options (1-4): "))

            if operation == 1:
                circular_enqueue()

            elif operation == 2:
                circular_dequeue()

            elif operation == 3:
                circular_display()

            elif operation == 4:
                circular_peek()

            elif operation == 5:
                break
```

```
            else:
                print("Invalid choice!")

    elif choice == 3: #If the user selects '3', the program will terminate
        print("\nProgram terminated.")
        break

    else:
        print("\nInvalid choice! Please try again.")
```

The outputs for this program are:

```
------------------------------

Program made by AMRAPALY SARKAR

------------------------------
1. Linear Queue
2. Circular Queue
3. Exit
Enter your choice: 1
```

```
---------- LINEAR QUEUE ----------
1. Insert element in the queue.
2. Delete element from the queue.
3. Display my queue
4. Show element at the front.
5. Back to Main Menu
What do you want to do in your queue? Choose from the options (1-4):
```

We are choosing to insert the element in our linear queue:

```
What do you want to do in your queue? Choose from the options (1-4): 1
Enter the values for your queue: 10
10 is now in the Queue.
```

The queue has the space to store upto 20 data elements.

When we perform deletion:

```
---------- LINEAR QUEUE ----------
1. Insert element in the queue.
2. Delete element from the queue.
3. Display my queue
4. Show element at the front.
5. Back to Main Menu
What do you want to do in your queue? Choose from the options (1-4): 2
10 is deleted from Queue.
```

The first element in the queue, i.e., 10, gets deleted from the linear queue from the front of the queue. This follows the principle of FIFO.

Now for Circular Queues:

Insertion:

```
--------- CIRCULAR QUEUE ---------
1. Insert an element in your circular queue.
2. Delete an element from your circular queue.
3. Display
4. Show front element in the circular queue.
5. Back to Main Menu
What do you want to do in your queue? Choose from the options (1-4): 1
Enter values for your queue: 20
20 inserted into Circular Queue.
```

20 is now in the circular queue. Let's add some more:

```
--------- CIRCULAR QUEUE ---------
1. Insert an element in your circular queue.
2. Delete an element from your circular queue.
3. Display
4. Show front element in the circular queue.
5. Back to Main Menu
What do you want to do in your queue? Choose from the options (1-4): 3

Your Circular Queue is: 20 30 40 50 50 60 70 80 90 99
```

Our circular queue will look something like this.

The best thing about this is, when the queue is completely full, and we delete some of the elements to accommodate the space for new insertions, they directly inserted into those empty spaces in the queue after deletion!

This saves us from a wastage in space when you want to insert a data element, but it doesn't get inserted, because it seems 'full', even when there are empty spaces in the front.

It gets rid of the wastage of space problem, that we would encounter in a Linear Data Structure.

Q.8. Write a program to perform the Infix to Postfix conversion using Python.

→The Infix Expression is the standard way of how people write an expression.

For example: (A*(B+C)) or (A + B)

Now, Postfix Expression is the way of writing an expression, where the operators are written after the operands.

For example: (AB+) is the same as (A +B).

The main difference between the Infix Expression and the Postfix Expression is where the operator is placed in the expression. If the operators are in between operands, then it is an Infix, while when the operators are placed after the operands, then it is a Postfix.

Here is a Python program for the Infix → Postfix Expression conversion:

main.py

```
print("\n ----------------------------------------")
print("\n This program is made by AMRAPALY SARKAR.")
print("\n ----------------------------------------")

def infix_to_postfix(expression):

    prec = {'+': 1, '-': 1, '*': 2, '/': 2} # Defining and storing the precedence of operators using a dictionary
    stack = [] # I am storing my operators in this stack
    output = [] # I am storing my alphabetical operands (A, B, c, etc.) or numerical operands (1, 2, and so on)

    for char in expression: # This whole loop for scanning the expression that will be inputted by the user

        if char.isalnum(): # If an operand is encountered during the scanning of the expression, it will pushed into the output stack
            output.append(char)

        elif char == '(': # If an opening bracket is encountered during the scan, it will be pushed into the stack
            stack.append(char)

        elif char == ')': # If a closing bracket is encountere
            while stack and stack[-1] != '(':
                output.append(stack.pop())
            stack.pop()  # Remove '(' from stack

        # If it's an operator (+, -, *, /) is encountered, push it into the stack, while checking the precedence.
        else:
            while (stack and stack[-1] != '(' and
                   prec.get(char, 0) <= prec.get(stack[-1], 0)):
                output.append(stack.pop())
            stack.append(char)

    # Popping any remaining operators left in the stack
    while stack:
        output.append(stack.pop())
```

```
    return "".join(output)

expr = input("Enter an infix expression (e.g., A+B*C): ") # Taking the input from the user

print(f"Infix Expression: {expr}") #The expressed inputted will be shown here
print(f"Postfix Expression: {infix_to_postfix(expr)}") #The conversion will be done and shown here
```

Let us see how it works:

```
 ----------------------------------------

 This program is made by AMRAPALY SARKAR.

 ----------------------------------------
Enter an infix expression (e.g., A+B*C): [A*(B+C)]
Infix Expression: [A*(B+C)]
Postfix Expression: ABC+*[]
```

You can clearly see the difference and the conversion. The Operands (A, B, C) were pushed into the Output, while the brackets [], () and Operators (*, +) were pushed into the Stack, when ever they were encountered in the iteration or scanning.

Here is another way of how Infix → Postfix works in Python:

main.py

```
print("\n ----------------------------------------")
print("\n This program is made by AMRAPALY SARKAR")
print("\n ----------------------------------------")

def infix_to_postfix(expression):
    # Define operator precedence
    precedence = {'+': 1, '-': 1, '*': 2, '/': 2, '^': 3}
    stack = []
    output = []

    for char in expression:
        # If the character is an operand, add it to the output
        if char.isalnum():
            output.append(char)
        # If the character is '(', push it to the stack
        elif char == '(':
            stack.append(char)
        # If the character is ')', pop from stack to output until '(' is encountered
        elif char == ')':
            while stack and stack[-1] != '(':
                output.append(stack.pop())
            stack.pop()  # Pop '(' from the stack
        # If an operator is encountered
        else:
            while (stack and stack[-1] != '(' and
                   precedence.get(char, 0) <= precedence.get(stack[-1], 0)):
                output.append(stack.pop())
            stack.append(char)

    # Popping all remaining operators from the stack
    while stack:
        output.append(stack.pop())
```

```
33
34      return "".join(output)
35
36  # Getting an expression from user input
37  infix_expr = input("Enter an infix expression (e.g., A*(B+C)/D): ")
38  postfix_expr = infix_to_postfix(infix_expr)
39
40  print(f"Infix Expression: {infix_expr}")
41  print(f"Postfix Expression: {postfix_expr}")
```

Let us see how this one works:

```
 ----------------------------------------

 This program is made by AMRAPALY SARKAR

 ----------------------------------------
Enter an infix expression (e.g., A*(B+C)/D): A+(B*C)^D
Infix Expression: A+(B*C)^D
Postfix Expression: ABC*D^+
```

Thus, this is how an expression is converted from Infix to Postfix using Python.

Q.9. Write a program in Python to perform Quick Sort.

→Quick Sort is a sorting algorithm that follows the Divide and Conquer concept. Which means that, to sort elements, the set of elements are divided into smaller parts to sort them, then put them all back together into a sorted order.

The program for Quick Sort in Python is as follows:

main.py

```
print("\n ---------------------------------------")
print("\n This program is made by AMRAPALY SARKAR")
print("\n ---------------------------------------")

def quick_sort(arr):
    # Base case: arrays with 0 or 1 element are already sorted
    if len(arr) <= 1:
        return arr

    # Choose the middle element as the pivot
    pivot = arr[len(arr) // 2]

    # Partition the array into three lists
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    # Recursively sort the sub-arrays and combine them
    return quick_sort(left) + middle + quick_sort(right)

# Get numbers from user input (e.g., typing: 38 27 43 3 9)
user_input = input("Enter numbers separated by spaces: ")

# Convert the space-separated input string into a list of integers
numbers = [int(x) for x in user_input.split()]

# Sort the array using quick sort
sorted_numbers = quick_sort(numbers)

print(f"Original Array elements: {numbers}")
print(f"Sorted Array elements: {sorted_numbers}")
```

Let's see how this works:

```
 ------------------------------------

 This program is made by AMRAPALY SARKAR

 ------------------------------------
Enter numbers separated by spaces: 10 34 56 73 12 9 56 8 23 109 32
Original Array elements: [10, 34, 56, 73, 12, 9, 56, 8, 23, 109, 32]
Sorted Array elements: [8, 9, 10, 12, 23, 32, 34, 56, 56, 73, 109]
```

See how the elements are sorted? This is done by Divide and Conquer principle used by Quick Sort. This is a quick and simple way for sorting a group of elements.

Another output:

```
 ------------------------------------

 This program is made by AMRAPALY SARKAR

 ------------------------------------
Enter numbers separated by spaces: 18 78 65 46 34 125 700 86 905 203 10 14 53 72 507
Original Array elements: [18, 78, 65, 46, 34, 125, 700, 86, 905, 203, 10, 14, 53, 72, 507]
Sorted Array elements: [10, 14, 18, 34, 46, 53, 65, 72, 78, 86, 125, 203, 507, 700, 905]
```

Thus, this is how Quick Sort works in Python.